

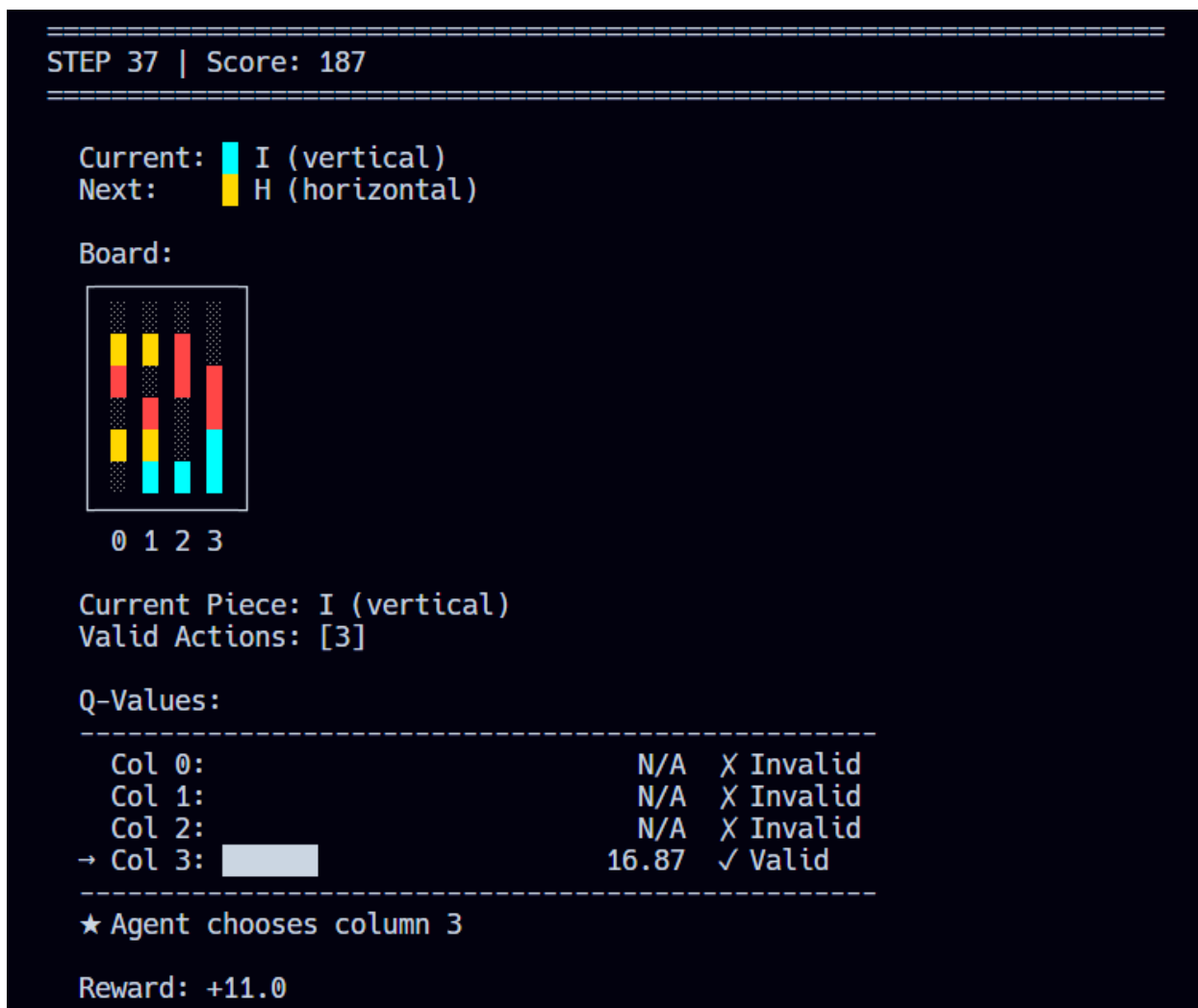
پروژه یادگیری تقویتی بخش اول (Markov Decision Process)

مبانی و کاربردهای هوش مصنوعی مهر ۱۴۰۴

در این پروژه قصد داریم دو الگوریتم Dynamic Programming برای حل مسائل MDP را پیاده‌سازی کنیم:

- Policy Iteration
- Value Iteration

و با استفاده از محیط tetris که در فایل های کد ارائه شده است، به بررسی تأثیر پاداش (Reward)، ضریب تنزیل (Discount Factor یا γ) و قطعیت (Deterministic) می پردازیم.

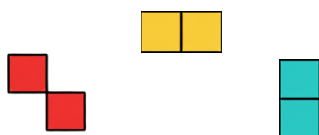


شکل ۱ - نمای بازی

توضیح خلاصه بازی :

تتریس یک بازی ویدئویی کلاسیک است که در آن قطعات هندسی (تکه‌ها) از بالای صفحه می‌افتند و بازیکن باید این قطعات را در جای مناسب قرار دهد تا ردیف‌های کامل ساخته شوند. وقتی یک ردیف کامل از قطعات پر می‌شود، آن ردیف پاک می‌شود و بازیکن امتیاز می‌گیرد. هدف بازی جلوگیری از پر شدن صفحه با قطعات است. اگر صفحه پر شود و دیگر فضای کافی برای قرار دادن قطعات جدید نباشد، بازی تمام می‌شود.

در بازی پیاده سازی شده ۳ شکل مختلف داریم که هر کدام به یک رنگ است.



- زرد : ۲ بلوک افقی
- آبی : ۲ بلوک عمودی
- قرمز : یک بلوک در سمت راست و ردیف پایین، یک بلوک سمت چپ و ردیف بالاتر

اعمال (Action Space)

قرار دادن اشکال در ۴ ستون مختلف در صورت برقرار بودن شرایط (خالی بودن)

فضای محیط (Observation Space)

```
def get_state(self)
    ...
    return tuple(heights + [self.current_piece])
```

در این محیط، حالت‌ها تاپلی تشکیل شده از ارتفاع‌های ستون‌ها $[h_0, h_1, h_2, h_3]$ و شکل فعلی است.

پاداش‌ها (Rewards)

پاداش‌ها بر اساس باختن یا خالی شدن سطر داده می‌شود.

به ازای خالی شدن هر سطر : +۱۰

به ازای باختن : -۲۰

جزئیات پیاده‌سازی

تمامی متدهای مورد نیاز برای ارتباط با محیط، از جمله مدیریت حالت‌ها، اعمال، پاداش‌ها و نمایش گرافیکی، در فایل‌های ارائه‌شده پیاده‌سازی شده‌اند.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $policy_stable \leftarrow true$
 For each $s \in \mathcal{S}$:
 $old_action \leftarrow \pi(s)$
 $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$
 If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$
 If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

شکل ۳ - سودوکد PI

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
 Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
 $\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

شکل ۵ - سودوکد الگوریتم VI

نکات تکمیلی

شما باید توابع `value_iteration` و `policy_evaluation` و `policy_improvement` و `policy_iteration` را در فایل `mdp.py` را تکمیل کنید.

دستورالعمل ارسال پروژه:

پس از تکمیل کد در فایل `mdp.py` و پاسخ به سؤالات تحلیلی در فایل ورد ارائه شده، کد به همراه گزارش تکمیل شده را در یک فایل زیپ با نام `FirstStudentName_SecondStudentName_MDP` قرار داده و آپلود کنید.

موفق باشید.